Reading the Architecture off the Weights: Manifest-Only Recovery of Neural Network Anatomy, with an Interactive Explorer

Sattaya Singkul
Researcher
joeysattaya@gmail.com

Abstract

To decide whether a model fits a device, how to quantize it, or simply what it is, practitioners need its architecture: which blocks repeat, what each block computes, how the components connect, and where the parameters sit. This information is scattered across model cards, family-specific configuration schemas and modeling code that must be read or executed. We show that most of it is already present in a much smaller artifact that almost every public checkpoint exposes: the *tensor manifest*, the list of tensor names, shapes and storage types that a safetensors file keeps in a header of a few tens of kilobytes. We present a method that turns a manifest into a structured anatomy. It normalizes quantized storage to logical tensors, finds repeated blocks in the module tree and types them by structural signature, assigns roles to the regions of each block from the algebra of their shapes, links components by matching residual widths, and finally fuses the fields that shapes cannot determine from `config.json` while checking that file against the weights. On 511 architectures instantiated from the `transformers` library, with ground truth taken from module classes the method never sees, per-tensor role accuracy is 96.4% (macro over architectures) against 69.2% for name-pattern rules, and 94.7% when every name is replaced by a random token. Logical parameter counts are exact for all models, also under NVFP4, FP8 and GPTQ storage. Shapes alone recover head geometry for about 72% of models; with configuration fusion this rises to 96%, and the consistency check finds that 95% of configurations agree with their weights, the rest being attention variants whose fields mean something else. The method runs in about 13 ms per model in a browser. We release it as an open, bilingual explorer that reads any Hugging Face repository over HTTP range requests, and we illustrate it on nine 2026 models, including a 121B hybrid Mamba–MoE vision-language model, two speech transducers and a video diffusion transformer.

1 Introduction

A deployment engineer handed a new checkpoint asks the same questions every time. How many layers are there, and are they all alike? Is it a mixture of experts, and how much of the size is experts? Which part runs once per input and which part runs once per generated token? Where will 4-bit quantization hurt first? For a familiar family, `config.json` answers these questions. For anything new it often does not. Configuration keys differ across families (`hidden_size`, `d_model`, `n_embd`); composite models nest several schemas; and fields such as `layers_block_type` only make sense after reading the modeling code. Reading that code is slow, and running it with `trust_remote_code` is a known supply-chain risk on public hubs [31]. Graph viewers such as Netron [22] or Model Explorer [7] need an exported graph and, in practice, the whole file.

A smaller artifact is almost always available. A safetensors file [13] begins with an 8-byte length and a JSON header listing every tensor's name, dtype, shape and byte offsets. Two HTTP range requests fetch it. Across the 511 architectures studied here its median size is 47 KB, about 6×10^{-5} of the weights in BF16

(Figure 6). PyTorch zip checkpoints carry the same facts in their `data.pkl` record, which can be read without touching tensor data.

This paper asks a simple question: *how much of a model's architecture can be read from its manifest alone, and how reliably?* The question matters in practice, because a tool that answers it can explain any hub model in a browser tab before anyone downloads it. It also has an analytical side. Some properties of an architecture leave no trace in tensor shapes, and an honest tool should say which ones, measure how often a prior gets them right, and fill them from a better source when one exists.

Our starting observation is that a manifest carries two kinds of evidence that complement each other. The *name tree* reveals structure: integer-indexed lists are stacks of blocks, identical subtrees are identical block types, and the module that owns a stack delimits a component. The *shapes* reveal function: every block shares a residual width d; an attention region is a set of projections that leave d and one that returns to it; a mixture of experts is a set of tensors whose leading dimension equals the output size of a router; a selective state-space mixer is a depthwise convolution next to a handful of per-head vectors; and two components are connected where the output width of one equals the input width of another. Names help break ties, but as we show, most of the work survives when every name is replaced by a random token.

Contributions.

- We formulate *manifest-only architecture recovery*, define an anatomy representation that is useful to people (components and data path, stacks and block types, regions and roles, logical sizes), and analyse which properties are not identifiable from shapes (§3).
- We give a recovery algorithm that combines module-tree structure with shape algebra and width matching, normalizes six quantized storage layouts, and fuses non-identifiable fields from `config.json` while checking the configuration against the weights (§4, §5).
- We build an open, bilingual explorer around the algorithm that reads any Hugging Face repository by range requests, shows every value with its provenance, and walks users through a model part by part (§6).
- We construct a benchmark of 511 architectures with exact manifests and independent ground truth, and answer five research questions on role recovery, structure, robustness, identifiability and cost (§7, §8). Case studies on nine 2026 models illustrate the coverage (§9).

2 Background and related work

How checkpoints describe themselves. A modern checkpoint usually ships three descriptions of the same model: the weights, a configuration file, and code. Model cards [16] add a human-written summary whose architectural fields are free text. The configuration is authoritative for the library that wrote it, but its schema is family-specific, and for new families it must be interpreted through code. Pickle-based weight formats can execute code on load [31]; safetensors avoids this, and its header is the most uniform description that exists across families.

Model visualization. Netron [22], TensorBoard and Model Explorer [7] render operator graphs from exported formats. They show what is in a graph faithfully but do not group operators into semantic parts, and they need the file. Transformer Explainer [2] explains one fixed architecture interactively and very well. Our explorer sits between these: semantic parts for arbitrary architectures, from a few kilobytes, with an explanatory walk-through borrowed from the second line of work.

Architecture inference. Side-channel attacks recover architectures from the cache timing or memory traces of a running model [12, 28]. There the observer lacks the file; here the header is public and the difficulty is interpretation. Weight-based fingerprinting such as HuRef [30] identifies a model's lineage from weight values; we never read values.

Architectures and storage formats. Our rules encode the shape signatures of attention and its variants (grouped-query [1] and latent attention [4]), sparse mixtures of experts [5, 24], selective state-space and linear-attention mixers [3, 9, 29] and their hybrids [17], conformers [10, 21], transducers [8, 27], diffusion transformers [19] and tabular in-context learners [11, 20]. For storage we handle GPTQ [6], AWQ [14], FP8 [15], MXFP4 [23], NVFP4 [18] and MLX layouts.

3 Problem formulation

3.1 Manifests and anatomies

A manifest is a set $\mathscr{M} = \{(n_i, s_i, \tau_i)\}$ of tensor names (dot-separated paths), shapes and storage dtypes. Optionally a configuration $\mathscr{C}$, a nested key–value tree, is also available. We want an *anatomy* $\mathscr{A}$ consisting of:

- **components** (for example a vision tower, a projector, a language backbone, a prediction network), each with a residual width d, a modality and a role, and directed **edges** for the data path;
- within each component, **stacks** of repeated blocks, each block assigned a **block type**;
- within each block type, **regions** labeled with a role from a fixed taxonomy: attention, cross-attention, feed-forward, mixture of experts, SSM or linear attention, convolution module, adaLN modulation, recurrent cell and norm; and outside blocks, input stem, output head and adapter;
- per-tensor roles (query, router, expert, ...), logical shapes under quantized storage, and derived quantities such as parameters, stored bytes, KV-cache bytes per token and active parameters per token;
- for every architectural field, its **provenance**: measured from the manifest, read from $\mathscr{C}$ and checked against the manifest, or estimated.

3.2 What a manifest cannot identify

Several properties leave no trace in shapes. (i) *Head count versus head size.* A projection $W_q \in \mathbb{R}^{Hh \times d}$ with H heads of size h has the same shape as one with $H/2$ heads of size $2h$. The pair is identifiable only through per-head parameters, such as query and key norms of width h or relative-position biases of shape $H \times h$; otherwise only the product Hh and the ratio of query to key–value width are. (ii) *Masking.* A decoder-only language model with tied embeddings stores exactly the tensors of a bidirectional encoder of the same size. (iii) *Routing and activation.* The number of experts is visible; the number used per token is not. Gated and ungated feed-forwards differ in matrix count, but SiLU and GELU do not. (iv) *Order inside a block.* Pre-norm and post-norm blocks have identical tensors.

These limits shape the design. The recovery algorithm reports such fields only as estimates, the explorer labels them as such, and when a configuration is available the fields are read from it and the configuration is checked against the shapes (§5.5). The evaluation measures both how often the estimate is right and how much fusion adds.

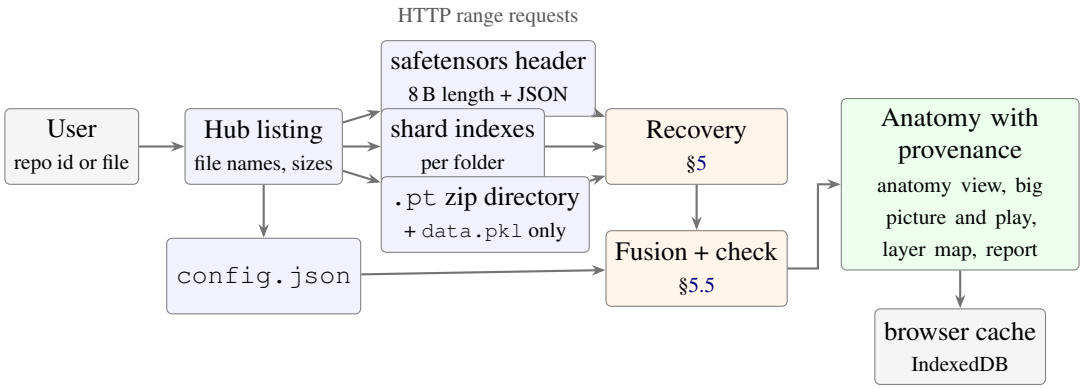


Figure 1: End-to-end workflow. Only descriptive bytes are fetched: the file listing, safetensors headers, shard indexes, the zip directory and pickle record of PyTorch checkpoints, and the configuration. Recovery and fusion run in the browser; results are cached locally so that a returning visitor does not repeat them.

4 System overview

Figure 1 shows the workflow. Given a repository id, the explorer lists the repository and decides what describes the weights. Safetensors files are preferred; their shard indexes are read per folder, so that a diffusers-style repository with `transformer/`, `text_encoder/` and `vae/` folders yields one prefix per component. For each safetensors file two range requests fetch the length and the header. If a repository has only PyTorch zip checkpoints, the reader fetches the last 64 KB of the file to locate the zip central directory, then fetches only the `data.pkl` record and interprets it with a restricted unpickler that rebuilds names, shapes and dtypes and never resolves a Python callable. The configuration is fetched alongside. Progress is shown at each step ("reading header 3 of 12, 41 KB so far"), and the analysed result is cached in the browser.

The manifest then passes through the recovery algorithm of §5, and the result through configuration fusion. Every field of the final anatomy carries its provenance, which the interface displays next to the value.

5 Recovering the anatomy

Every rule in this section reads only shapes, dtypes and the structure of the name tree. A small lexicon of name fragments (`attn`, `mlp`, `experts`, ...) is consulted to break ties that shapes leave open and can be switched off; §8 measures the method with and without it. Algorithm 1 summarizes the procedure.

5.1 Normalizing storage

Quantized checkpoints store one logical weight as several tensors. We group tensors by module and recognize the layout by dtype and shape before looking at names. NVFP4 stores a `U8` tensor of shape $[o, i/2]$, an FP8 scale tensor $[o, i/16]$ and a scalar second-level scale; the logical shape is $[o, i]$ and the group size follows from the ratio. FP8 stores an FP8 weight with a scalar, per-channel $[o, 1]$ or block scale. GPTQ stores `qweight` as `I32` $[i/8, o]$ with scales $[i/g, o]$; AWQ packs the other axis, and the two are told apart by which axis the scales share. MXFP4 and MLX layouts follow the same pattern. Auxiliary tensors count toward stored bytes but not toward parameters.

Algorithm 1 Manifest-only anatomy recovery with configuration fusion

Require: manifest $\mathscr{M}$, optional configuration $\mathscr{C}$
$\mathscr{T} \leftarrow$ NORMALIZE($\mathscr{M}$) ▷ logical shapes, stored bytes
build the module tree; find lists, expert lists, sub-layer lists, block lists, homes
for all block list L **do**
    group items by structural signature into block types; record the type sequence
    for all block type β **do**
        $d \leftarrow$ width of the leaf norms of a representative block
        classify children by shape rules; split nested blocks; pair halves; test transposition
    end for
end for
label stems and heads; detect adapters, transducer joints, timestep MLPs, message passing
choose the backbone and family; derive d, L, f, E, V, KV bytes per token
if $\mathscr{C}$ is given **then**
    match sub-configurations to components by width and depth; read H, H_{kv}, h, top-k
    check $Hh = x$ and $H_{kv}h = x_{kv}$; label each field *config* or report the mismatch
end if
return anatomy with provenance and an evidence string for every decision

5.2 Blocks from the module tree

Names form a tree. A node whose children are all integers, or keys such as `expert_0`, `expert_1`, ..., is a *list*. An item is *composite* if its subtree holds at least two parameterized modules; this separates blocks from `nn.Sequential` wrappers around single layers.

Expert lists are lists nested inside the item of another list whose items are structurally identical two- or three-matrix MLPs without norms, usually beside a router whose first dimension equals the number of items. Stacked experts instead appear as 3-D tensors $[E, \cdot, \cdot]$ with one trailing dimension equal to the block width.

Block lists are the innermost composite lists that are not expert lists, with one correction: when every item of an outer list holds one or more short inner lists of equal length (T5's `block.`i`.layer.`j, or LongCat's two attention and two MLP sub-layers per layer), the inner items are sub-layers and the outer items are the blocks.

Each block list belongs to a *home*, the module that owns it, raised through its ancestors while no other stack lives there and, in multi-folder repositories, never above the folder. Homes become components, so embeddings and heads next to an `encoder` wrapper join the right component. Inside a list, each block receives a signature: the sorted multiset of pairs (relative path with indices masked, tensor rank). Identical signatures form a block type. In a hybrid stack such as Nemotron-H [17] this yields one type for each of Mamba-2, MoE and attention layers, and the sequence of types is the layer pattern.

5.3 Regions by shape algebra

For each block type we analyse one representative block. Its residual width d is the length of the vectors held by its leaf norm modules; this is more robust than counting matrix dimensions, which an SSM's inner width can dominate. The children of the block are then classified. Let a matrix be an *input* if its second dimension is d and an *output* if its first dimension is d.

- **Mixture of experts:** stacked 3-D experts or an expert list, with a router $[E', d]$ where $E \le E' \le E + 8$ (some routers reserve extra slots).
- **SSM / linear attention:** a depthwise convolution $[C, 1, k]$ with $k \le 8$ plus recurrence parameters attached directly to the mixer: at least two vectors of a common length $n_h \neq d$ (for A, D, the Δt bias), or

a 2-D or 3-D state tensor with a small last dimension. Mixers without a convolution (xLSTM, RWKV) are recognized by two or more per-head gate projections $[\leq 64, d]$ or by decay vectors.

- **Feed-forward:** one output $[d, f]$ and one or two inputs $[f, d]$, or one fused input $[2f, d]$ with $f > 1.3d$.
- **Attention:** one output $[d, x]$ with one or more inputs, or one fused input $[x + 2x_{kv}, d]$. Inputs whose own input width is produced inside the region are internal; this keeps the low-rank chains of latent attention [4] inside one region. A region whose external inputs include d and at least two projections from another width $w \geq 128$ is *cross-attention* from the component of width w. Vectors in norm-only submodules whose length divides x give the head size; parameters of shape $[H, h]$ with $Hh = x$ give both factors.
- **Recurrent cell:** two or more matrices whose rows are $3h$ or $4h$ (GRU, LSTM), none returning to d.
- **Convolution module:** a depthwise convolution with pointwise $[C, C', 1]$ convolutions or batch-norm statistics.
- **adaLN modulation:** a single matrix $[kd, t]$ from a width $t < d$, or $k \in \{6, 9, 12, 18\}$ when $t = d$, or a $[1, k, d]$ scale–shift table.

Three refinements matter in practice. A child holding two non-leaf sub-modules that classify into different confident roles (an attention and an MLP inside one "inducing block") is split. Two half feed-forwards in sibling modules ($[f, d]$ in `intermediate` and $[d, f]$ in `output`) are paired. If a region is weak in the usual layout but becomes attention when every matrix is transposed, the weights are stored as $[\text{in}, \text{out}]$, as GPT-2's `Conv1D` does.

5.4 Stems, heads and the data path

Outside blocks, a matrix $[V, d]$ with $V > d$ is a token table and its twin an untied output head; a vector of length V is the head's bias; a small table beside a token table or convolutional stem is a position or type embedding. A matrix chain whose free input width equals one component's width d_X, or $k^2 d_X$ for $k \in \{2, 3, 4\}$ after pixel shuffling, and whose free output width equals another component's d_Y, becomes an *adapter* with edges $X \to \text{adapter} \to Y$.

Three further patterns become components of their own. A *transducer* has a label embedding $[V, h_p]$ next to a recurrent cell, an encoder projection $[j, d_{\text{enc}}]$, and an output $[V', j]$ with $V \leq V' \leq V + 16$; the excess $V' - V$ is the number of duration bins of a token-and-duration transducer [27]. A *diffusion denoiser* has a timestep MLP from a sinusoidal width (128, 256 or 320) to a width t that feeds the modulation matrices in its blocks. A group of square maps $[C, C]$ with an aggregation matrix $[C, rC]$ is a message-passing layer with r reducers.

The backbone is the largest component that owns a token table or output head. The family (decoder, prefix LM, encoder–decoder, transducer, diffusion denoiser, encoder) follows from the components and edges.

5.5 Configuration fusion and consistency checking

When a configuration is available, the fields of §3.2 are read from it. Configurations of composite models nest several sub-configurations, so we first decide which sub-configuration describes which component. Every object node of $\mathscr{C}$ that declares a width (`hidden_size`, `d_model`, `n_embd`, `dim`, ...) is a candidate; a component is matched to the candidate whose width equals its residual width, preferring one whose layer count equals the component's and whose path names the right kind of component (`text`, `llm`, `vision`, `audio`). From the match we read the head count, key–value head count, head size and routing top-k.

The match is then checked against the weights: the product of head count and head size must equal the width x of the attention projections, and the key–value product must equal x_{kv}. A field that passes is labeled

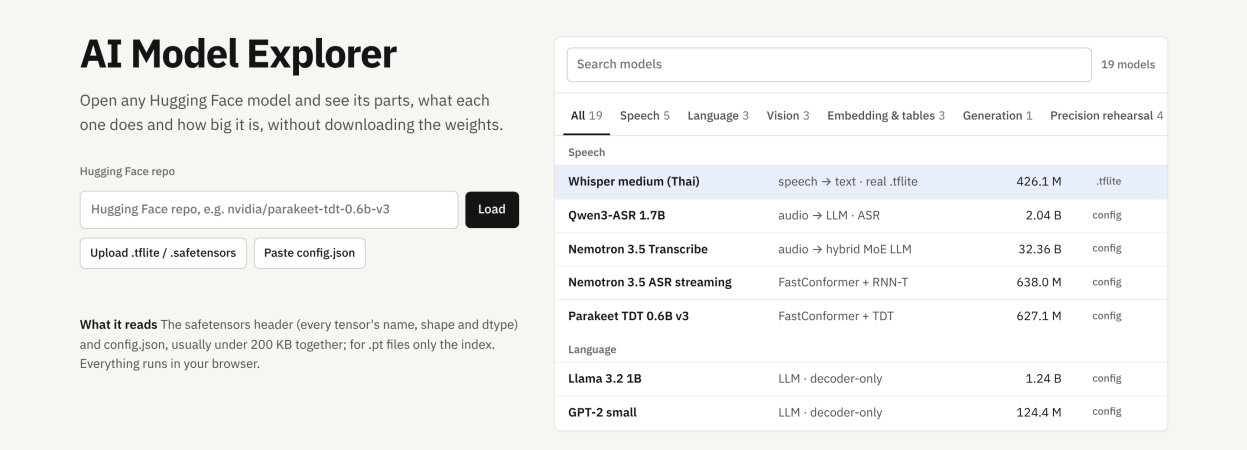


Figure 2: Entry screen. A repository id, an uploaded `.safetensors` or `.tflite` file, or a pasted configuration starts an analysis; the library on the right holds analysed templates grouped by task, with search, and the models a visitor opened before.

config; a disagreement is reported rather than resolved, because it usually means that a field carries a different meaning in that family (§8.4). Without a configuration, the head size falls back to a prior (128 for $d \geq 2048$, otherwise 64) and is labeled *estimate*.

5.6 Derived quantities and cost

Parameters and stored bytes are sums over logical and stored tensors. KV-cache bytes per token are $2 \sum_{\ell \in \text{attn}} x_{kv}^{(\ell)}\, b$; SSM layers keep a constant state instead. With a top-k value, active parameters per token are the non-expert parameters plus k/E of the experts. The procedure makes a few passes over the tree and is near-linear in the number of tensors.

6 The interactive explorer

The algorithm is the core of an open web tool [25]. We built it for a hands-on workshop on edge inference, where participants had to understand a speech model before compressing it, and then generalized it to any architecture.

Design goals. Three goals guided the interface. First, *answer the first question in one screen*: what the model is made of and how big each part is. Second, *show where each value comes from*, so that a reader can distinguish a measurement from a configuration value and from an estimate. Third, *explain by walking*, letting a newcomer step through the model from input to output, seeing what each part receives and produces, instead of watching every layer animate.

Views. The entry screen (Figure 2) offers three ways in and a library of analysed models grouped by task. The *anatomy view* (Figure 3) draws the components in data-path order. Each box shows the component's role, parameters, stored size, width and depth, and a strip with one cell per layer coloured by block type, so a hybrid backbone reads at a glance. Selecting a component lists its block types with the share of parameters per region and the shape evidence behind each label. A panel on the right lists the values read from the model, each tagged *measured*, *config* or *estimated*, and shows whether the configuration agrees with the weights.

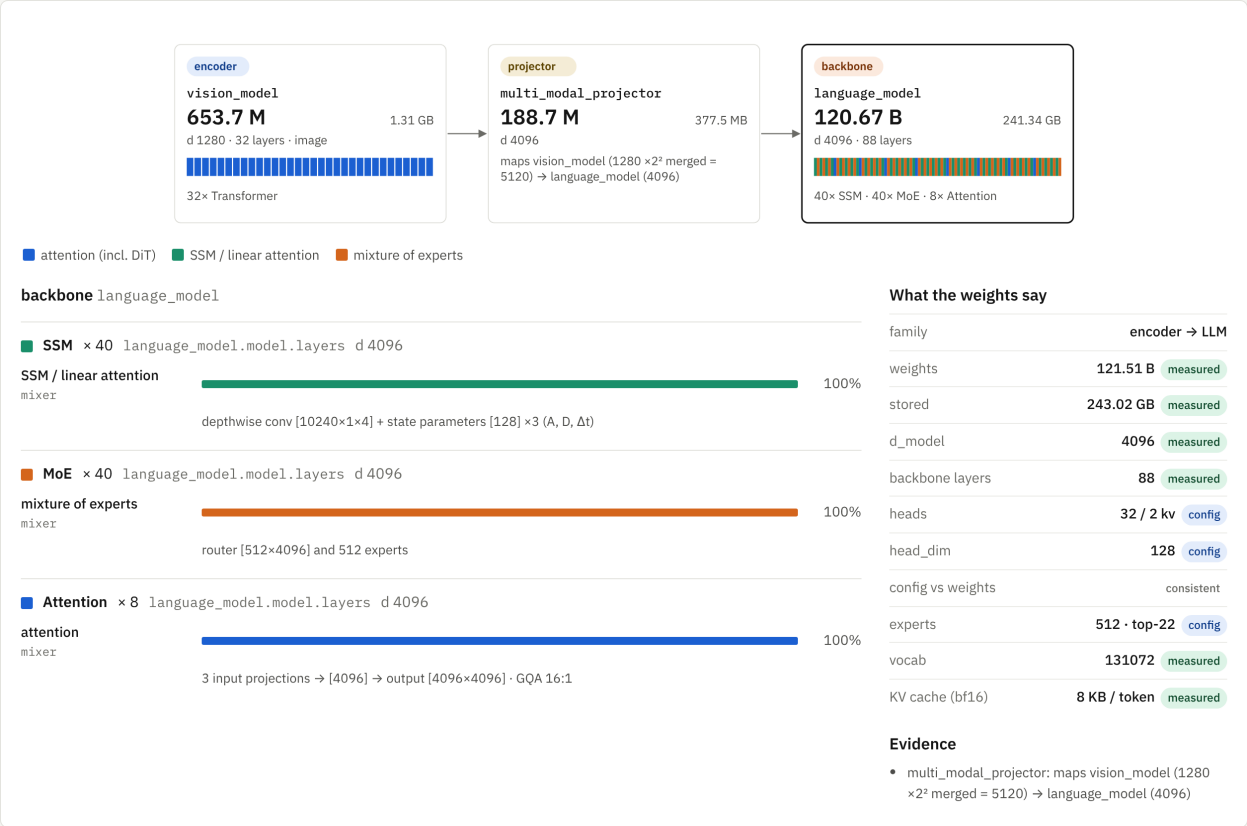


Figure 3: Anatomy view for Nemotron 3.5 Super VL (121.5 B). Vision tower, projector and backbone in data-path order; the backbone's strip shows the interleaving of SSM (green), MoE (orange) and attention (blue) layers. Below, each block type with the share of parameters per region and the evidence behind its label; on the right, the values read from the model with their provenance and the result of the configuration check.

The *big picture* (Figure 4) lays the model out as columns from input to output. Pressing *Play* steps through the columns; for each one a panel shows what goes in and what comes out, with small visual stand-ins (a waveform, a spectrogram, a token row, a matrix), the tensor shapes, and one sentence on why the part matters for optimization. The stack column zooms once into a single layer to show its internal steps, and the loop at the end is drawn according to the family: next token for decoders, next frame for transducers, next denoising step for diffusion models. A *layer map* draws every layer at its true size under switchable lenses (size, bytes read per sentence, precision), and a *report* lists where the parameters, compute and memory traffic go, with automatically generated observations ("experts hold 92.8% of weights; about 13.6 B are active per token"). The interface is available in Thai and English.

A usage walkthrough. Suppose a user wants to deploy `nvidia/parakeet-tdt-0.6b-v3` on a phone. They paste the repository id and press *Load*. The explorer lists the repository, fetches `config.json` and the 63 KB header of the 2.5 GB weight file, and finishes in well under a second on a normal connection. The anatomy view shows three components: a 24-block conformer encoder holding 97% of the parameters, a 12 M prediction network with a two-layer LSTM, and a 6 M joint network whose output size, 8198, is the label vocabulary plus five duration bins. The block view shows that feed-forward modules hold two thirds of each conformer block, which suggests where pruning or low-bit storage pays. The play-through then explains that the encoder runs once per utterance, whereas the prediction and joint networks run per emitted label, so latency in streaming use is governed by the small networks, not by the encoder.

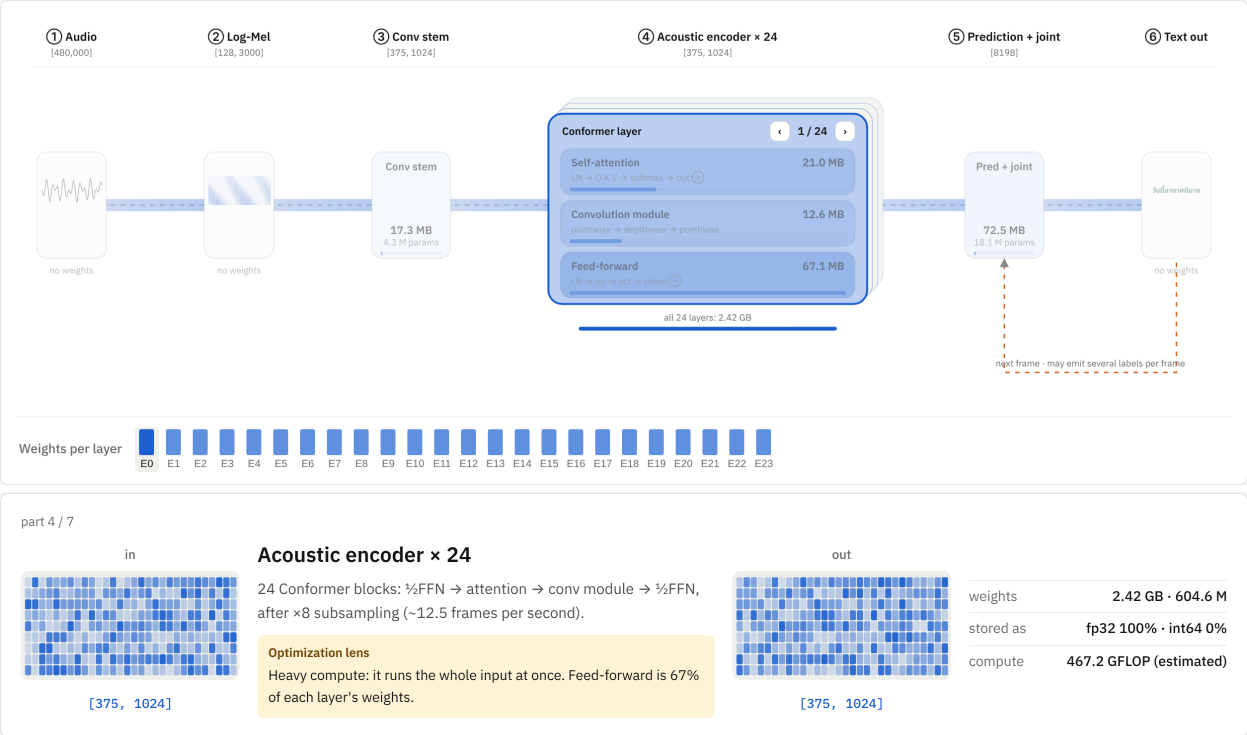


Figure 4: Big picture and play-through for Parakeet TDT 0.6B v3. Top: the transducer laid out from audio to text, with the prediction and joint networks as one part and the per-frame loop drawn back to them. Bottom: the panel for step 4 of 7, the acoustic encoder, with its input and output shapes, stored size, precision and estimated compute.

Implementation. The tool is a single static page with no server component, about 3,000 lines of JavaScript, and runs offline once loaded. The recovery code is a separate module shared with the evaluation harness, so the benchmark measures exactly the code users run. Analysed models are stored in the browser's IndexedDB.

7 Evaluation setup

We ask five research questions. **RQ1** How accurately are per-tensor roles recovered, and how much depends on names? **RQ2** Are stacks, block counts and block types recovered? **RQ3** Is recovery robust to quantized storage? **RQ4** Which hyperparameters can shapes identify, how much does configuration fusion add, and do configurations agree with their weights? **RQ5** What does recovery cost?

Corpus. `transformers` 5.22.1 [26] registers 788 configuration types. For each, we instantiate the default configuration with the most specific model class available on PyTorch's meta device, which creates shapes without allocating memory. 548 types instantiate; removing identical manifests leaves 511. Tied parameters are counted once and persistent buffers are kept, as a saved checkpoint would. The corpus has 268,040 tensors (median 419 per model) and spans 1.5 K to 1.74 T parameters (median 473 M). Of the 511 models, 475 contain attention, 73 mixtures of experts, 27 SSM or linear-attention mixers, 140 convolutional parts and 70 cross-attention; 231 have more than one stack and 153 have a stack with more than one block type. The configuration of each model is serialized with it for **RQ4**.

Ground truth. Labels come from information the algorithm never sees: the Python class of every module. A tensor's role is given by its nearest ancestor class whose name ends in a recognized suffix (`Attention`,

	Role accuracy (%)		Block lists (%)				Params
Condition	macro	micro	recall	precision	count	types	exact
Names + shapes (ours)	**96.4**	**99.9**	98.6	91.4	100	92.5	511/511
Shapes only (ours)	94.7	99.8	98.6	91.4	100	92.5	511/511
Name patterns	69.2	8.4	–	–	–	–	–
Name patterns, names hashed	0.0	0.0	–	–	–	–	–
Ours, NVFP4 storage	96.4	99.9	98.6	91.4	100	92.5	511/511
Ours, FP8 storage	96.4	99.9	98.6	91.4	100	92.5	511/511
Ours, GPTQ storage	96.4	99.9	98.6	91.4	100	92.5	511/511
Ours, shapes only + NVFP4	94.7	99.8	98.6	91.4	100	92.5	511/511

Table 1: Main results on 511 architectures. *count*: matched lists with the exact number of blocks; *types*: matched lists whose block-type partition equals the ground truth. The name-pattern baseline produces no stacks. Micro accuracy is dominated by the three largest models, all trillion-parameter mixtures of experts, whose experts we label correctly and the baseline does not.

`MLP`, `SparseMoeBlock`, `Mamba2Mixer`, `RMSNorm`, ...), with MoE containers overriding their children and a short list of module-name fallbacks for loose tensors; tensors that match nothing are labeled *other* and excluded. Convolutional stems and position tables outside block stacks are merged into one *input stem* role, because the library is inconsistent about which class owns them. Block lists are module lists whose children each hold at least two parameterized sub-modules, excluding expert containers and applying the same sub-layer convention as the algorithm; block types are the distinct class-composition signatures of the items. Hyperparameters come from the configuration, using the text sub-configuration for multimodal models.

Metrics. Per-tensor role accuracy is weighted by parameter count within each model and averaged over models (*macro*, our headline, so that every architecture counts once) or pooled over all tensors (*micro*, which the largest models dominate). We also report per-role F1 over tensors; recall and precision of block lists by path; the share of matched lists with the exact block count and with the exact type partition; exact-match accuracy of hyperparameters; and exactness of the logical parameter total.

Conditions and baseline. *Names + shapes* is the full method. *Shapes only* replaces every non-numeric path token by a consistent random hash, removing all lexical evidence while keeping the tree; it approximates checkpoints exported by unfamiliar code. *Quantized* rewrites every 2-D linear weight as NVFP4 (group 16), FP8 with per-channel scales, or GPTQ 4-bit with group 128 (Appendix B). The *baseline* is a careful set of name-pattern rules of the kind most tools use, taken from the previous version of our own explorer: regular expressions for layer indices, `q_proj`/`query`/`c_attn`, `mlp`/`fc1`/`intermediate`, `encoder_attn`, `mixer`, and so on.

8 Results

8.1 RQ1: Roles

Per-tensor role accuracy is 96.4% macro (Table 1). The distribution over models is skewed: the median model is labeled perfectly, 411 of 511 reach 99% and 468 reach 90%, while 10 fall below 50%. Name-pattern rules reach 69.2% macro; their micro accuracy collapses to 8.4% because the corpus's parameter mass sits in a few trillion-parameter MoE defaults whose experts they mislabel. With names hashed the baseline has

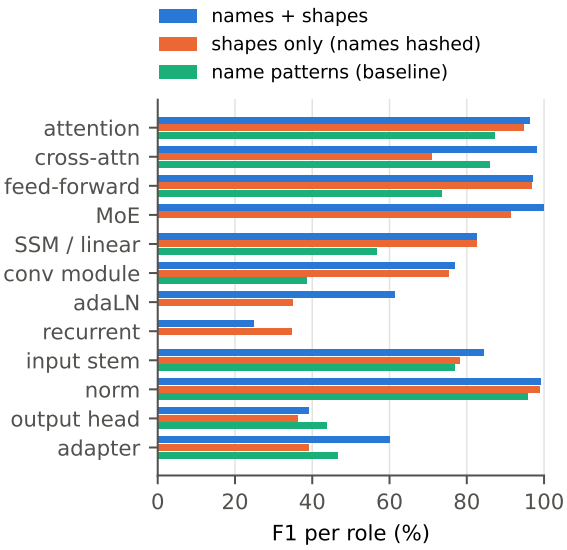


Figure 5: Per-role F1 over tensors.

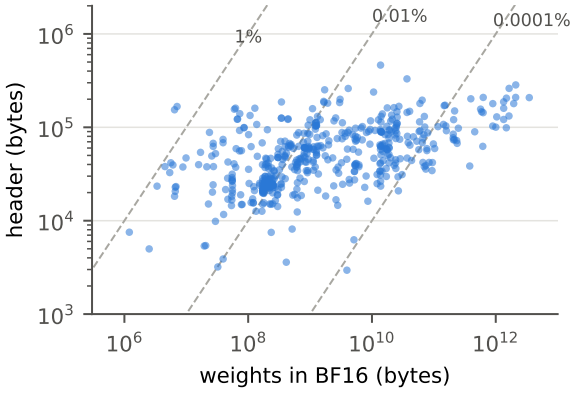


Figure 6: Header size against weight size for the 511 manifests (one 3 KB model lies off the left edge). Dashed lines mark constant ratios; the median header is 47 KB and the largest 453 KB.

nothing to work with, whereas our method loses 1.7 points (371 models still reach 99%). Most of the work is therefore done by structure and shapes, not vocabulary.

Figure 5 breaks the result down by role. Attention, feed-forward, MoE and norms exceed 96 F1 with names and stay above 91 without them. Cross-attention falls from 98 to 71 without names, which is expected: when encoder and decoder share a width, the two attention regions of a decoder block have identical shapes, and only a name says which reads the encoder. adaLN modulation (4 models) and recurrent cells (6 models) have too few examples for stable F1. Output heads score lowest, mostly because pooler and prediction-transform layers are labeled *head* by the class oracle but look like a feed-forward or an embedding to the shape rules.

The largest error groups are informative. ConvBERT's span-based dynamic convolution sits inside its attention module, so the region is labeled a convolution. The linear-attention layers of GLM-5-Next keep ordinary query, key and value projections and move the decay parameters into a sub-module, so the shape rules see attention; the opposite happens in EdgeTAM's perceiver, whose gated attention looks recurrent. ALBERT shares one layer across depth and stores no list at all. I-BERT keeps integer copies of every weight whose matching shapes imitate a gate stack, which also produces the only false transducer detection in the corpus; the three real transducers are found. The ten models below 50% (ALBERT, I-BERT, xLSTM, XLNet, MobileBERT, MobileViTv2, PoolFormer, PatchTSMixer, NeoMME and an ASR encoder with fused blocks) each depart from the block conventions the rules assume rather than failing on a common architecture.

8.2 RQ2: Stacks and block types

98.6% of ground-truth block lists (1232 of 1250) are found at the right path, and every matched list has the right number of blocks. Precision is lower (91.4%) because the algorithm also reports repeated sub-blocks inside convolutional backbones, such as ResNet bottleneck sub-layers and SEANet residual units, that the ground-truth convention does not count. The block-type partition is exact for 92.5% of matched lists; most misses are lists whose first block differs only in a module the class signature sees and the tensor signature

	d	layers	FFN	experts	vocab	heads	KV heads	head size
Manifest only, all models	92.7	92.1	90.5	97.3	87.2	72.2	70.0	66.7
n	468	442	336	74	360	446	227	153
Manifest only, single stack	95.0	96.3	94.1	96.4	85.6	70.3	63.9	57.1
Manifest only, shapes only	92.7	92.1	90.5	94.6	84.7	72.2	70.0	66.7
With configuration fusion	–	–	–	–	–	96.0	98.2	96.7
Identifiable from shapes?	yes	yes	yes	yes	yes	only with per-head parameters		

Table 2: Exact-match accuracy (%) of hyperparameters against the configuration. FFN is evaluated on dense models only. Fusion is evaluated on the 510 models whose configuration serialized.

does not, such as a relative-position bias owned by layer 0.

8.3 RQ3: Quantized storage

Under NVFP4, FP8 and GPTQ storage the results equal those at full precision, and logical parameter totals remain exact for all 511 models (Table 1). This holds with names hashed as well, because packed weights and their scales are recognized by dtype and shape ratios rather than by suffixes such as `weight_scale`. The point has practical weight: hub pages count packed bytes as parameters, so an NVFP4 model is often displayed at about 60% of its real size (§9).

8.4 RQ4: Identifiability, configuration fusion and consistency

Table 2 separates identifiable from non-identifiable fields. Width, depth, FFN size and expert count are read correctly for 90–97% of models, and more often when a model has a single stack, which removes ambiguity about which component the configuration describes. From shapes alone, head count and head size are right about 70% of the time. The prior is right more often (78%) than a norm vector of plausible length (69%): in latent attention, the norm before the low-rank KV projection has the latent width, not the head width. This is a concrete instance of the identifiability limit of §3.2.

With configuration fusion, head count, KV head count and head size are right for 96.0%, 98.2% and 96.7% of models. Because the ground truth comes from the same configuration, this number measures the *matching* step, that is, whether the right sub-configuration is attached to the backbone, rather than independent accuracy; the backbone found a match in 456 of 510 models and 874 of 1108 components found one. The more informative result is the consistency check. Of the 635 matched components with attention, 603 (95.0%) have configurations whose head fields multiply out to the projection widths in the weights. The 32 disagreements are concentrated in latent-attention models (DeepSeek-V2/V3/V4, Kimi, MiniCPM3, Mistral 4, LongCat-Flash), where `head_dim` or `num_key_value_heads` refers to the query–key or latent geometry rather than to a projection width, in Falcon's fused multi-query layout, and in SAM-style windowed vision encoders. None of them is a wrong configuration; each is a field whose meaning is family-specific. A tool that silently trusted the configuration would report wrong KV-cache sizes for exactly these models, which is why the explorer shows the mismatch instead of resolving it.

8.5 RQ5: Cost

The median safetensors header is 47 KB, the 95th percentile 164 KB and the largest 453 KB, for models up to 3.5 TB in BF16 (Figure 6). Recovery takes 12.6 ms for the median model in a single JavaScript thread (95th percentile 63 ms, maximum 177 ms). In the explorer, network round trips dominate: a single-file model needs four requests (listing, configuration, two range requests), and a sharded model two range requests per shard.

Model	Family	Params	Header	Recovered anatomy (selected)
Nemotron 3.5 Super VL 120B-A12B	prefix LM	121.5 B	130 KB	ViT tower (32 L, d 1280) $\rightarrow$ projector $1280\times2^2{=}5120 \rightarrow 20480 \rightarrow 4096$ $\rightarrow$ 88-layer backbone: 40 SSM, 40 MoE (512 experts, 92.8% of weights), 8 attention (2 KV heads, 8 KB KV per token)
Nemotron 3.5 Transcribe 30B-A4B	prefix LM	32.4 B	154 KB	audio encoder (32 L, d 1280), diarization encoder (31 L, d 512), word-timestamp encoder, projector $1280 \rightarrow 2688$, 52-layer backbone: 23 SSM, 23 MoE (128 experts), 6 attention; 4.36 B active with top-6
Nemotron 3.5 ASR streaming 0.6B	transducer	638 M	57 KB	24 conformer blocks; LSTM prediction net; joint $1024,640 \rightarrow 640 \rightarrow 13088$, no duration bins (RNN-T)
Parakeet TDT 0.6B v3	transducer	627 M	63 KB	same encoder shape; joint output $8198 = 8193 + 5$ duration bins (TDT); total within 24 of the hub's figure
Nemotron 3 Embed 1B NVFP4	encoder	1.14 B	41 KB	16 blocks, d 2048, 24/8 heads (config, consistent), 112 NVFP4 linear layers (group 16), tied table; stored 1.03 GB
FastH3 Omni FP8 (transformer)†	diffusion	33.1 B	84 KB	2 refiner + 50 DiT blocks, 56 heads of 128 (measured from q/k norms), adaLN $18d$ (6 terms $\times$ 3 modalities), timestep MLP $256 \rightarrow 5376 \rightarrow 2688$; 300 FP8 tensors, 47.0 vs. 66.3 GB
Kumo Tabular (large)†	encoder	214 M	41 KB	row embedding (6 induced + 6 row blocks, d 256) and 24-layer ICL backbone (d 1024); 854 MB in FP32 vs. 855 MB file
Kumo Relational†	encoder	29 M	24 KB	row embedding $\rightarrow$ message passing (4 square maps, 5 reducers) $\rightarrow$ 12-layer ICL backbone; 118 vs. 120 MB file

Table 3: Anatomies recovered for 2026 models (the BF16 FastH3 variant gives the same structure). Manifests marked † are rebuilt from public source code and configuration because no reference implementation is in `transformers`; the others are generated from the published `config.json` by the reference implementation, with NVFP4 storage following the published quantization config.

9 Case studies: nine models from 2026

These models were not in the benchmark, but they were not held out from development either: we studied them while extending the rules, and several parts of §5 (transducer joints, timestep MLPs, message-passing layers, adapter components) were written because of them. Every change was checked on the full benchmark for regressions. The case studies therefore show that the representation is rich enough for these architectures; the unbiased accuracy estimate is §8. Table 3 summarizes them.

A 121B hybrid. The Super VL backbone mixes three block types in one list, and the recovered type sequence reproduces the configuration's `layers_block_type` exactly. The projector's input width, 5120, is four times the vision width; the algorithm reports a 2×2 merge, which matches the configuration's `downsample_ratio` of 0.5. Experts hold 112.7 B of 121.5 B parameters. With the configured top-22 routing that gives 13.6 B active parameters per token, including the vision tower and both embedding tables, against the 12 B in the model's name. Only 8 of 88 layers keep a KV cache, each with 2 KV heads, so the cache costs 8 KB per token in BF16; if all 88 layers used the same attention it would be eleven times larger.

Transducers. For both speech models the joint network is found from three shapes, and the number of duration bins falls out as a difference: Parakeet's joint output has 8198 rows and its label table 8193, giving the five durations $\{0,\ldots,4\}$ of its configuration, while the streaming model's two sizes are equal (a standard RNN-T). Parakeet's parameter total differs from the hub's by 24, the number of batch-norm step counters, one per conformer block.

Packed weights and encoders. The NVFP4 embedding model has 1.14 B logical parameters, matching its model card, while the hub's file statistics report 0.7 B because they count packed `U8` elements. That it is an encoder cannot be seen in the weights; its configuration's `is_causal: false` settles it, and the explorer labels the family as coming from the configuration.

What the case studies taught us. Besides the new components, four fixes to existing rules came from these models: the residual width of a block is taken from its norms; `nn.Sequential` children are no longer split as sub-layers; a module wrapping two different sub-blocks is split; and in repositories with several top-level folders, homes are not raised above the folder. The streaming ASR model's language-prompt projector remains unattached, because its prompt table lives outside the manifest's components.

10 Discussion and limitations

Ground truth. Class names are an independent signal but not a perfect one. Some implementations place a position table inside a class called `...Embeddings` and others inside the encoder; some call a pooler a `...Head`. We merged roles where the library is inconsistent and kept the fallback list short, but per-role numbers for heads and adapters should be read with this in mind. The headline numbers are robust to it because these roles carry little parameter mass.

One library's conventions. All corpus manifests come from `transformers` implementations, which share naming habits. The shapes-only condition is the main protection against overfitting to those habits, and its 1.7-point drop suggests the structural rules transfer. Default configurations are not trained checkpoints, but their shapes, which are all the method reads, are the same as those of trained ones.

Configuration fusion. Fusion accuracy is measured against the same configuration that supplies the fields, so it validates the matching step, not the configuration. The consistency check is the independent part, and its disagreements point at fields whose meaning differs by family; a richer check could model latent attention explicitly.

Reconstructed manifests and scope. Three case-study manifests were rebuilt from source; their totals match the published file sizes within 2%, but module prefixes in the real checkpoints may differ. The explorer reads the real headers when given a repository. GGUF and ONNX files are not handled yet, although both carry comparable metadata; old non-zip PyTorch pickles are rejected. Order inside a block and activation functions remain unknown, and FLOP estimates assume standard computation per role.

Responsible use. The method reads only what a hub already serves publicly. It never downloads or executes weights, and the PyTorch reader interprets a fixed subset of pickle opcodes as data without resolving Python callables.

11 Conclusion

A tensor manifest is small, safe and almost always available, and it carries more of an architecture than one might expect. With the module tree, a few dozen shape rules and width matching, we recover blocks, block types, regions, data paths and logical sizes for 511 architectures at 96% role accuracy, keep most of that accuracy without any names, and stay exact on parameter counts under modern quantized formats. The same analysis makes the limits explicit: head geometry, masking and routing are not in the shapes. Fusing them from the configuration closes most of the gap, and checking the configuration against the weights exposes the families whose fields mean something else. We hope the benchmark and the explorer make it routine to look inside a model before downloading it.

Availability. The explorer runs at `https://joessattes.github.io/ai-model-explorer/`. Source code, the corpus builder, the evaluation harness and this paper's sources are at `https://github.com/JoesSattes/ai-model-explorer` under the MIT license. The results in §8 are reproduced by `python research/build_corpus.py corpus.jsonl` followed by `node research/eval.js corpus.jsonl` with the flags `-obf -nolex`, `-quant nvfp4|fp8|gptq`, `-v1` and `-cfg`.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[2] Aeree Cho, Grace C. Kim, Alexander Karpekov, Alec Helbling, Zijie J. Wang, Seongmin Lee, Benjamin Hoover, and Duen Horng Chau. Transformer explainer: Interactive learning of text-generative models. *arXiv preprint arXiv:2408.04619*, 2024.

[3] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning*, 2024.

[4] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[5] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[6] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.

[7] Google AI Edge. Model explorer: Intuitive and hierarchical visualization of model graphs. `https://ai.google.dev/edge/model-explorer`, 2024.

[8] Alex Graves. Sequence transduction with recurrent neural networks. *arXiv preprint arXiv:1211.3711*, 2012.

[9] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[10] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. Conformer: Convolution-augmented transformer for speech recognition. In *Interspeech*, 2020.

[11] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637:319–326, 2025.

[12] Xing Hu, Ling Liang, Shuangchen Li, Lei Deng, Pengfei Zuo, Yu Ji, Xinfeng Xie, Yufei Ding, Chang Liu, Timothy Sherwood, and Yuan Xie. Deepsniffer: A dnn model extraction framework based on learning architectural hints. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 385–399, 2020.

[13] Hugging Face. safetensors: Simple, safe way to store and distribute tensors. https://github.com/huggingface/safetensors, 2022.

[14] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, 2024.

[15] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.

[16] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019.

[17] NVIDIA. Nemotron-H: A family of accurate and efficient hybrid Mamba-Transformer models. *arXiv preprint arXiv:2504.03624*, 2025.

[18] NVIDIA. Pretraining large language models with NVFP4. *arXiv preprint arXiv:2509.25149*, 2025.

[19] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

[20] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. TabICL: A tabular foundation model for in-context learning on large data. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 50817–50847, 2025.

[21] Dima Rekesh, Nithin Rao Koluguri, Samuel Kriman, Somshubra Majumdar, Vahid Noroozi, He Huang, Oleksii Hrinchuk, Krishna Puvvada, Ankur Kumar, Jagadeesh Balam, and Boris Ginsburg. Fast conformer with linearly scalable attention for efficient speech recognition. In *IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2023.

[22] Lutz Roeder. Netron: Visualizer for neural network, deep learning and machine learning models. https://github.com/lutzroeder/netron, 2017.

[23] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

[24] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

[25] Sattaya Singkul. AI Model Explorer. https://joessattes.github.io/ai-model-explorer/, 2026.

[26] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.

[27] Hainan Xu, Fei Jia, Somshubra Majumdar, He Huang, Shinji Watanabe, and Boris Ginsburg. Efficient sequence transduction by jointly predicting tokens and durations. In *International Conference on Machine Learning*, 2023.

[28] Mengjia Yan, Christopher W. Fletcher, and Josep Torrellas. Cache telepathy: Leveraging shared resource attacks to learn DNN architectures. In *29th USENIX Security Symposium*, pages 2003–2020, 2020.

[29] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, 2025.

[30] Boyi Zeng, Lizheng Wang, Yuncong Hu, Yi Xu, Chenghu Zhou, Xinbing Wang, Yu Yu, and Zhouhan Lin. HuRef: HUman-REadable fingerprint for large language models. In *Advances in Neural Information Processing Systems*, 2024.

[31] Jian Zhao, Shenao Wang, Yanjie Zhao, Xinyi Hou, Kailong Wang, Peiming Gao, Yuanchao Zhang, Chen Wei, and Haoyu Wang. Models are codes: Towards measuring malicious code poisoning attacks on pre-trained model hubs. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.

A Corpus construction details

The model class for each configuration type is chosen in this order: image-text-to-text, multimodal LM, speech sequence-to-sequence, sequence-to-sequence LM, TDT, RNN-T, causal LM, CTC, masked LM, image classification, text-to-waveform, object detection, base model. Instantiation uses `_from_config` under `torch.device("meta")` with a 90-second limit. Of the 240 types that do not instantiate, 177 are sub-configurations without a model class of their own (vision or audio towers registered separately), and the rest fail on required sub-configurations or optional dependencies. Manifests are deduplicated by the exact list of names and shapes.

B Synthetic quantized storage

For every 2-D tensor named `*.weight` whose input dimension is a multiple of 32 and output dimension a multiple of 8, excluding embeddings, heads and norms: NVFP4 replaces $[o, i]$ by `U8` $[o, i/2]$, `F8_E4M3` $[o, i/16]$ and two `F32` scalars; FP8 by `F8_E4M3` $[o, i]$ and `F32` $[o, 1]$; GPTQ by `qweight I32` $[i/8, o]$, `qzeros` $[i/128, o/8]$, `scales F16` $[i/128, o]$ and `g_idx` $[i]$. Under name hashing these suffixes are hashed too, so recognition relies on dtypes and shape ratios.

C Additional interface views

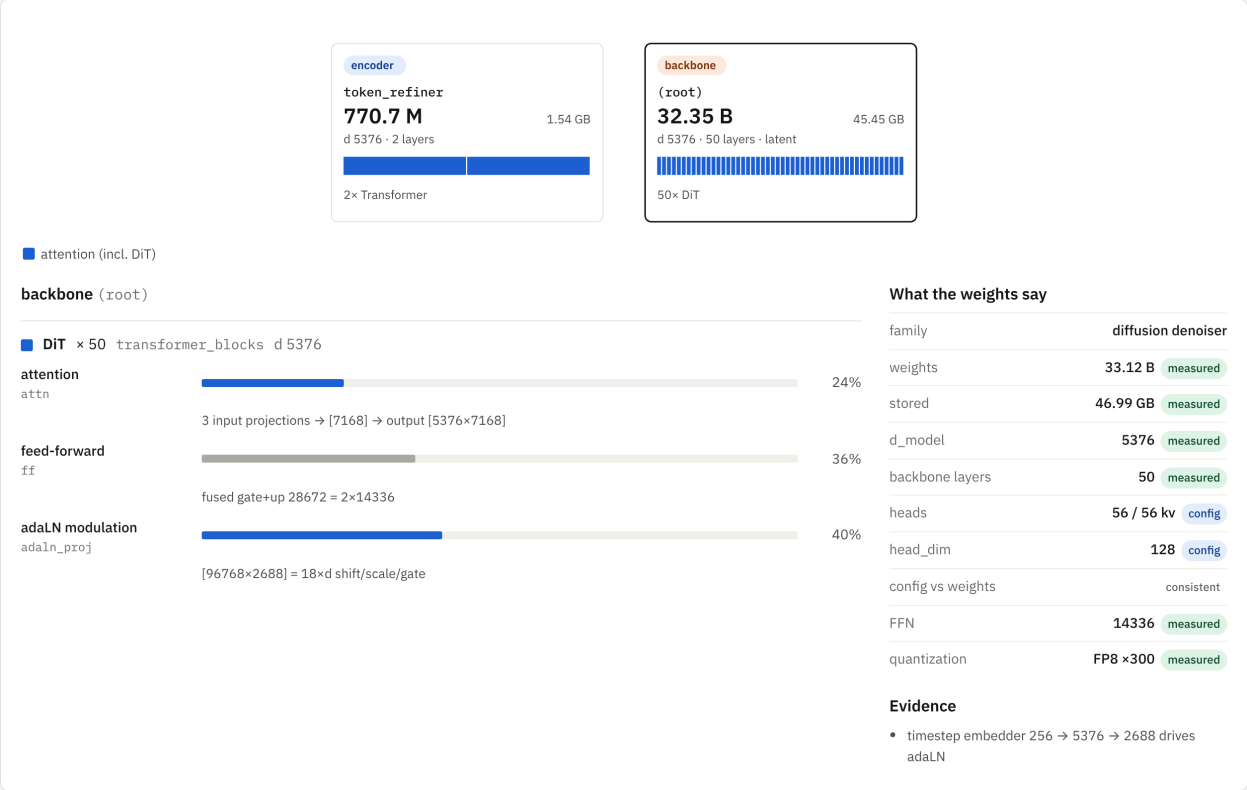


Figure 7: Anatomy view for the FastH3 Omni FP8 denoiser (reconstructed manifest): a two-block token refiner and a 50-block DiT backbone. The head geometry is *measured* from per-head norms, and the FP8 storage of 300 tensors is reported with logical and stored sizes.